

Verifikation verteilter Systeme in Isabelle: Ein Vergleich von Kleppmann und I/O-Automaten

Sepp Stage (Matrikelnummer: 791122)
Technische Universität Chemnitz
Hauptseminar
Sommersemester 2026
Betreuer: M. Sc. Jonas Henschel
21. Juni 2026

Inhaltsverzeichnis

1	Einleitung	4
2	Das Zwei-Phasen-Commit-Protokoll	5
2.1	Problemstellung	5
2.2	Rollen	5
2.3	Grundlegender Ablauf	5
2.4	Protokollverhalten bei Fehlern	6
2.4.1	Fehlerarten in verteilten Systemen	6
2.4.2	Teilnehmerfehler beim Zwei-Phasen-Commit-Protokoll	6
2.4.3	Netzwerkfehler beim Zwei-Phasen-Commit-Protokoll	6
2.5	Eigenschaften	7
3	Grundlegende Konstrukte in Isabelle	8
4	Kleppmann	10
4.1	Modellbeschreibung	10
4.2	Übergangsfunktionen für das Zwei-Phasen-Commit-Protokoll	12
4.3	Invarianten	13
4.4	Beweis der Invarianten	15
4.4.1	Hilfslemma für Koordinator	17
4.4.2	Hilfslemma für Teilnehmer	17
4.5	Beweis der einheitlichen Zustimmung	19
5	I/O Automaten	24
5.1	Mathematische Beschreibung nach Lynch	24
5.1.1	Ausführung eines IOA	24
5.1.2	Komposition	25
5.2	I/O Automaten in Isabelle	25
5.3	Übergangsrelationen für das Zwei-Phasen-Commit-Protokoll	27
5.4	Invarianten	30
5.5	Beweis der Invarianten	30
5.6	Beweis der einheitlichen Zustimmung	35
6	Vergleich der Modellierungsansätze	36
6.1	Erarbeitung der Vergleichskriterien	36
6.2	Flexibilität und Modularität	36
6.3	Ähnlichkeit Formalisierung vs Algorithmus	37
6.4	Formalisierung des Zwei-Phasen-Commit-Protokolls	37
6.5	Beweisstruktur	37
6.6	Beweisaufwand	38
6.7	Nachweis von Liveness-Eigenschaften	38
6.8	Automatisierungsgrad in Isabelle	39
6.9	Dokumentation zur Nutzung der Frameworks	39

7	Schluss	40
8	Quellen	41

1 Einleitung

Im Januar 1990 kam es zu einem Fehler im AT&T Kommunikationsnetz, der über 60.000 Amerikaner ohne Telefonanschluss ließ. Dieser großflächige Ausfall wurde durch eine einzelne Netzwerkkomponente in New York ausgelöst [1].

Der Ausfall und anschließende Neustart dieser Komponente führte zu Nachrichten, die von benachbarten Netzwerkkomponenten nicht korrekt gedeutet wurden, was ebenfalls zum Absturz dieser Komponenten führte [2]. So konnte sich ein lokales Problem über das System großflächig ausbreiten.

In verteilten Systemen können selbst kleine Probleme einen großen Schaden anrichten. Die Schwierigkeit besteht darin, dass kleine und seltene Fehlerkombinationen oft durch Tests alleine nicht vollständig abgedeckt werden können. Bereits kleine unvorhergesehene Abweichungen im Nachrichtenverhalten können schnell zu großflächigen Ausfällen führen.

Diese Komplexität verteilter Systeme erfordert daher formale Methoden mit mathematischen Beschreibungen, welche maschinelle Verifikation ermöglichen. Dadurch kann unter den getroffenen Annahmen Korrektheit mathematisch nachgewiesen werden.

Ein bekanntes Beispiel für die Schwierigkeit verteilter Koordination ist das Zwei-Phasen-Commit-Protokoll, welches Konsistenz zwischen mehreren Teilnehmern garantiert, jedoch empfindlich gegenüber Ausfällen ist [3].

Ziel dieser Arbeit ist die formale Modellierung und Verifikation des Zwei-Phasen-Commit-Protokolls im Beweisassistenten Isabelle. Dabei werden zwei unterschiedliche Modellierungsansätze untersucht. Eine Modellierung basierend auf dem verteilten Systemmodell nach Kleppmann, und eine Darstellung mittels I/O-Automaten nach Lynch und Tuttle. Das Zwei-Phasen-Commit-Protokoll wird in beiden Frameworks implementiert. Ebenfalls wird dabei die Eigenschaft der einheitlichen Zustimmung formalisiert und bewiesen. Da beide Modellierungsansätze Vor- und Nachteile hinsichtlich Nutzbarkeit, Beweisaufwand und Modularität aufweisen, werden die Implementierungen und Beweise anschließend im jeweiligen Framework analysiert und miteinander verglichen.

2 Das Zwei-Phasen-Commit-Protokoll

Dieses Kapitel beschreibt das zugrunde liegende verteilte Protokoll unabhängig von seiner formalen Darstellung. Hier wird die von Jens Lechtenbörger formalisierte Version des Protokolls zusammengefasst dargestellt [4].

2.1 Problemstellung

Das Zwei-Phasen-Commit-Protokoll wird in verteilten Systemen eingesetzt, um alle teilnehmenden Prozesse zu einer einheitlichen Entscheidung zu führen. Dies kann beispielsweise eine Entscheidung darüber sein, ob eine Datenbanktransaktion akzeptiert und dauerhaft oder abgelehnt und rückgängig gemacht werden soll.

2.2 Rollen

Das Protokoll unterteilt die teilnehmenden Prozesse in zwei Arten. Es gibt einen Koordinator, der die letztendliche Entscheidung trifft. Der Rest der Prozesse sind die Teilnehmer [4]. Im Folgenden bezeichnet 'Prozess' einen allgemeinen Teilnehmer in einem verteilten System, während der Begriff 'Teilnehmer' speziell für einen nicht-Koordinator Prozess im Zwei-Phasen-Commit-Protokoll verwendet wird.

2.3 Grundlegender Ablauf

Der Ablauf des Protokolls wird hier anhand einer durchzuführenden Datenbanktransaktion beschrieben. Ein Übergangsdiagramm inklusive Verhalten bei Fehlern, wird in Abbildung 1 beschrieben.

Das Protokoll unterteilt sich in zwei Phasen. Die erste Phase ist die Vorbereitungsphase, wobei alle Prozesse im Initial-Zustand starten. Der Koordinator beginnt durch das Senden einer Prepare-Nachricht an alle Teilnehmer und wechselt in den Collecting-Zustand. Beim Erhalt einer solchen Nachricht wertet der Teilnehmer lokal aus, ob er in der Lage ist, die Transaktion durchzuführen. Ist dies der Fall, so schickt er eine Yes-Nachricht an den Koordinator zurück und wechselt in den Prepared-Zustand. Andernfalls sendet er eine No-Nachricht und geht in den Aborted-Zustand über.

Der Koordinator sammelt nun die Antworten der Teilnehmer und entscheidet daraufhin den Ablauf der zweiten Phase des Protokolls. Erhält er eine No-Nachricht, so schickt er eine Abort-Nachricht an alle Teilnehmer und wechselt in den Aborted-Zustand. Diese antworten daraufhin mit einer Ack-Nachricht und wechseln ebenfalls in den Aborted-Zustand. Sollte er stattdessen von allen Teilnehmern eine Yes-Nachricht erhalten haben, so schickt er eine Commit-Nachricht an alle Teilnehmer und wechselt in den Committed-Zustand. Diese antworten wiederum mit Ack-Nachrichten und wechseln ebenso in den Committed-Zustand. Sobald der Koordinator von allen Teilnehmern eine Ack-Nachricht erhalten hat, geht er in den Forgotten-Zustand über und das Protokoll ist beendet.

2.4 Protokollverhalten bei Fehlern

In verteilten Systemen können verschiedenste Arten an Fehlern auftreten. Diese müssen korrekt behandelt werden, um auch beim Auftreten dieser Fehler einen korrekten Ausgang des Protokolls zu gewährleisten [5]. Dabei ist ebenfalls entscheidend, welche Fehler als möglich angenommen werden. Diese Annahmen werden als Systemannahmen bezeichnet und definieren den formalen Rahmen, innerhalb dessen ein Algorithmus modelliert und analysiert werden kann. Im Folgenden wird eine Zusammenfassung möglicher Fehlerarten in verteilten Systemen gegeben. Das Verhalten der Prozesse kann Abbildung 1 entnommen werden.

2.4.1 Fehlerarten in verteilten Systemen

Jeder Prozess in einem verteilten System kann auf verschiedenste Art und Weise mit oder ohne Warnung abstürzen [6], wobei entweder nur jegliche nicht persistente Daten verloren gehen oder gar ganze Datensätze korrumpiert werden können. Dieser Absturz kann entweder final sein oder der Teilnehmer kann erneut gestartet werden [3].

Teilnehmer können auch byzantinisches (willkürliches) Verhalten zeigen, wobei dieses Verhalten von fehlerhaften Nachrichtensendungen bis hin zu gezielt böartigen Handlungen reicht [3].

Nicht nur die Teilnehmer in einem verteilten System können fehlerhaft sein, auch das zugrunde liegende Netzwerk kann Fehler aufweisen. So kann eine einzelne gesendete Nachricht verloren gehen, dupliziert werden oder eine beliebig lange Zulieferzeit aufweisen [3]. Im schlimmsten Fall kann eine Nachricht beim Empfänger verändert ankommen, als es beim Sender abgeschickt wurde. Dabei kann zwischen erkennbar und unerkennbar geänderten Nachrichten entschieden werden. Mehrere Nachrichten können zusätzlich untereinander umgeordnet werden, sodass sie beim Empfänger in einer anderen Reihenfolge ankommen als sie beim Sender abgeschickt wurden [5].

2.4.2 Teilnehmerfehler beim Zwei-Phasen-Commit-Protokoll

Die hier betrachtete Version des Zwei-Phasen-Commit-Protokolls [4] betrachtet nur Abstürze von Koordinator und Teilnehmern, bei denen der persistente Speicher erhalten bleibt und lediglich nicht persistenter Speicher vollständig verloren geht. Um nach einem solchen Absturz das Protokoll korrekt fortsetzen zu können, schreibt jeder Prozess vor dem Senden einer oder mehrerer Nachrichten seinen aktuellen Zustand in den persistenten Speicher. Nach einem Absturz kann dann anhand des zuletzt geschriebenen Zustands dieser aus dem Speicher geladen werden. Dabei müssen jedoch die in diesem Zustand zu sendenden Nachrichten erneut gesendet werden.

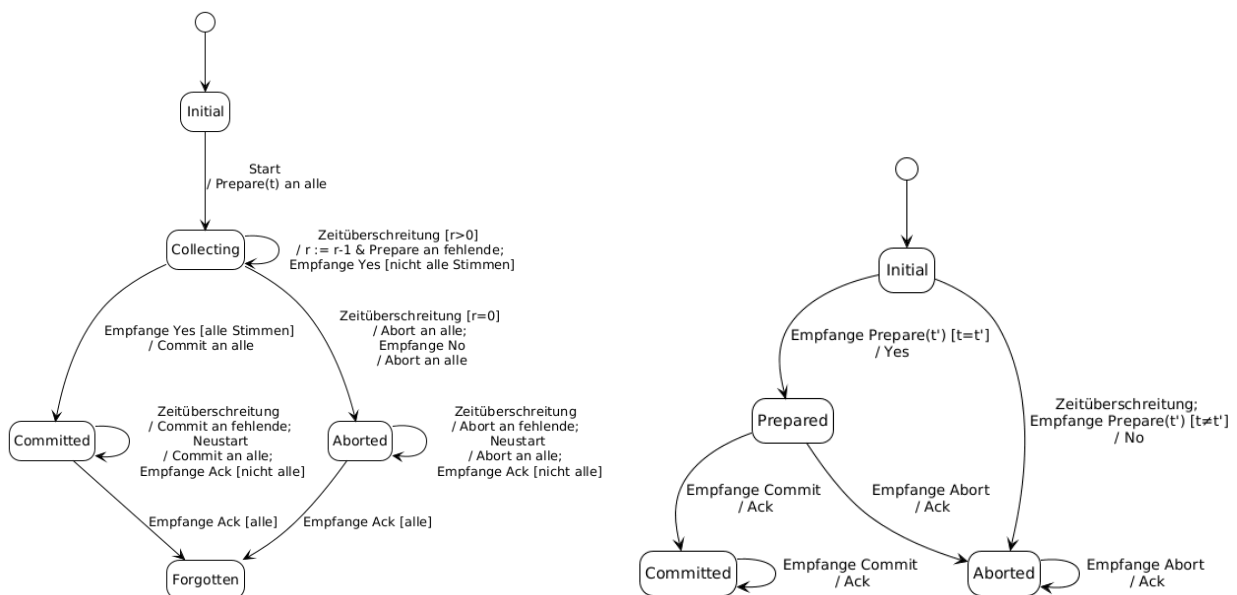
2.4.3 Netzwerkfehler beim Zwei-Phasen-Commit-Protokoll

Neben den beschriebenen Teilnehmerausfällen behandelt dieses Zwei-Phasen-Commit-Protokoll auch Fehler im Kommunikationsnetz. Gesendete Nachrichten können verloren

gehen [4]. Um damit korrekt umzugehen, wird bei jedem Senden einer oder mehrerer Nachrichten ein lokaler Timer gestartet der, sobald er abläuft, eine Sendungswiederholung jener Nachrichten auslöst. Zusätzlich, sollten wiederholt die Timer des Koordinators beim Warten im Collecting-Zustand ablaufen, so bricht dieser die Entscheidung eigenständig ab und sendet Abort-Nachrichten an alle Teilnehmer.

Sollte nach einer gewissen Zeit keine Prepare-Nachricht bei einem Teilnehmer ankommen, so wechselt dieser eigenständig in den Aborted-Zustand, ohne eine Nachricht zu senden. Bei der Implementierung dieses Protokolls werden später zusätzlich Nachrichtenduplizierung, Nachrichtenverspätung und Umordnung der Nachrichten betrachtet.

Abbildung 1: Übergangsdiagramm für Koordinator (links) und Teilnehmer (rechts)



2.5 Eigenschaften

Eine der wichtigsten Eigenschaften, die das Zwei-Phasen-Commit-Protokoll zu jeder Zeit erfüllen muss, ist die 'einheitliche Zustimmung' [3]. Sie besagt, dass sich alle Prozesse, die sich entscheiden, zur gleichen Entscheidung kommen müssen [7]. Diese Eigenschaft muss zu jeder Zeit der Ausführung des Protokolls gelten. Dies führt dazu, dass sobald sich alle Prozesse entschieden haben, eine insgesamt einheitliche Entscheidung vorliegt. Weitere Eigenschaft wie, dass sich alle nicht abgestürzten Prozesse irgendwann entscheiden wird hier nicht weiter betrachtet.

3 Grundlegende Konstrukte in Isabelle

Isabelle ist ein interaktiver Theorembeweiser zur formalen Verifikation mathematischer Beweise. Ebenfalls können Modelle und Programme spezifiziert werden [5]. Dieses Kapitel führt die notwendigen Isabellekonzepte ein.

Funktionen können auf drei verschiedene Weisen definiert werden. Isabelle unterstützt die Lambda Notation

```
1      (λx. x)
```

welche besonders nützlich zur ad-hoc Definition von Funktionen ist. Nicht rekursive Funktionen können mittels Definitionen wie folgt definiert werden:

```
1      definition sq1 :: "nat ⇒ nat" where  
2      "sq1 n = n*n"
```

Rekursive Funktionen müssen auf folgende Weise definiert werden:

```
1      fun sq2 :: "nat ⇒ nat" where  
2      "sq2 0 = 0" |  
3      "sq2 (Suc n) = sq2 n + 2*n + 1"
```

Beweise können in Isabelle wie folgt geführt werden:

```
1      lemma sq1_eq_sq2:  
2      assumes "sq1 n = s1"  
3      and "sq2 n = s2"  
4      shows "s1 = s2"  
5      using assms  
6      proof(induction n arbitrary: s1 s2)  
7      case 0  
8      then show ?case  
9      by (simp add: sq1_def)  
10     next  
11     case (Suc n)  
12     then show ?case  
13     using sq1_def by fastforce  
14     qed
```

Wobei neben `simp` und `fastforce` beliebige Beweistaktiken in Verbindung mit beliebigen Aussagen, wie hier `sq1_def`, vorkommen können [8].

Ein wichtiger Datentyp in Isabelle sind Listen. Die Leere Liste wird durch `[]` notiert. Zwei Listen können durch den `@`-Operator aneinander angefügt werden [8].

Mengen können wie folgt definiert werden:

```
1      {} (*Leere Menge*)  
2      {t | x y. P}
```


wobei x und y die freien Variablen im Term t sind, die in P vorkommen. [8].

Auf Paare kann mittels der Projektionsfunktionen fst und snd auf das jeweils erste und zweite Element des Paares zugegriffen werden. Tupel werden als nach rechts geschachtelte Paare betrachtet. [9]. Konditionale Anweisungen werden wie folgt geschrieben:

```
1      if P then A else C
```

oder

```
1      case t of  
2      A => B |  
3      C => D |  
4      _ => E
```

Diese Sprachkonstrukte bilden die Grundlage für die späteren Implementierungen.

4 Kleppmann

Dieses Kapitel beschreibt den ersten Modellierungsansatz, welcher von Martin Kleppmann beschrieben wurde [5]. Sein Framework, samt eines Beispielprojekts kann online gefunden werden [10].

4.1 Modellbeschreibung

Im Modell von Martin Kleppmann wird ein verteiltes System als eine Übergangsfunktion dargestellt, welche Ereignisse verarbeitet [5]. Events werden als eigener Datentyp dargestellt, welcher einen Konstruktor für jede Eventart beinhaltet. Dieser Datentyp muss für jedes Protokoll angepasst werden, um zu definieren, was einem Prozess passieren kann.

Für das hier betrachtete Zwei-Phasen-Commit-Protokoll wird dieser Datentyp wie folgt definiert:

```
10 datatype ('proc, 'msg) event
11   = Start
12   | Receive (msg_sender: 'proc) (recv_msg: 'msg)
13   | Timeout
14   | Restart
```

Das Start-Event kann als Beginn des Protokolls verstanden werden. Das Receive-Event dient zum Empfangen von Nachrichten. Die Timeout- und Restart-Events dienen zum Darstellen von ablaufenden Timern, für gesendete Nachrichten und darstellen von Neustarts abgestürzter Prozesse.

Der Typ einer Übergangsfunktion wird explizit definiert, um das Framework übersichtlicher zu halten.

```
16 type_synonym ('proc, 'state, 'msg) step_func =
17   "'proc ⇒ 'state ⇒ ('proc, 'msg) event ⇒ ('state × ('proc, 'msg) send
   ↪ set)"
```

Eine solche Übergangsfunktion nimmt den Prozess der das Event verarbeitet, dessen aktuellen Zustand und das zu verarbeitende Event an und gibt ein Tupel bestehend aus dem neuen Zustand des Prozesses und eine Menge, der von diesem Prozess, in diesem Schritt gesendete Nachrichten zurück [5]. Eine gesendete Nachricht besteht dabei aus Empfängerprozess und Nachrichtentyp und hat folgende Form:

```
7 datatype ('proc, 'msg) send
8   = Send (msg_recipient: 'proc) (send_msg: 'msg)
```

Weiterhin wichtig ist zu definieren, wann welches Event auftreten kann. Diese Semantik wird in folgender Funktion beschrieben:

```

19 fun valid_event :: "('proc, 'msg) event  $\Rightarrow$  'proc  $\Rightarrow$ 
20   ('proc  $\times$  ('proc, 'msg) send) set  $\Rightarrow$  bool" where
21   "valid_event Start _ _ = True" |
22   "valid_event (Receive sender msg) proc msgs = ((sender, Send proc msg)
23      $\hookrightarrow$   $\in$  msgs)" |
24   "valid_event _ _ _ = True"

```

Dabei wird in Zeile 22 festgelegt, dass keine Nachrichten im Netzwerk grundlos entstehen können [5]. So kann ein Receive-Event vom Empfänger einer Nachricht nur auftreten wenn diese entsprechende Nachricht vorher gesendet wurde.

Um einen speziellen Zeitpunkt während der Ausführung darzustellen, gibt es folgende induktive Definition:

```

25 inductive execute ::
26   "('proc, 'state, 'msg) step_func  $\Rightarrow$  ('proc  $\Rightarrow$  'state)  $\Rightarrow$  'proc set  $\Rightarrow$ 
27   ('proc  $\times$  ('proc, 'msg) event) list  $\Rightarrow$ 
28   ('proc  $\times$  ('proc, 'msg) send) set  $\Rightarrow$  ('proc  $\Rightarrow$  'state)  $\Rightarrow$  bool" where

```

Sie erhält als Eingabe eine Übergangsfunktion für die Prozesse, eine Initialisierungsfunktion für deren Startzustände, die Menge der beteiligten Prozesse, eine Historie der bisher verarbeiteten Ereignisse (inklusive zugehöriger Prozesse), die Menge aller gesendeten Nachrichten sowie eine Zustandsfunktion für die aktuellen Prozesszustände und gibt zurück, ob diese eine valide Ausführung darstellen.

Der Basisfall der induktiven Definition stellt die leere Ausführung ohne Nachrichten und Ereignisse dar:

```

29 "execute step init procs [] {} init" |

```

Zuletzt wird beschrieben, wie sich die Nachrichten und Ereignisse verändern müssen, damit eine valide Ausführung auch nach einem verarbeiteten Ereignis valide bleibt:

```

30 "[execute step init procs events msgs states;
31   proc  $\in$  procs;
32   valid_event event proc msgs;
33   step proc (states proc) event = (new_state, sent);
34   events' = events @ [(proc, event)];
35   msgs' = msgs  $\cup$  (( $\lambda$ msg. (proc, msg)) ` sent);
36   states' = states (proc := new_state)
37 ]  $\Rightarrow$  execute step init procs events' msgs' states'"

```

Das Framework stellt drei häufig verwendete Hilfslemmas bereit, die zentrale Eigenschaften gültiger Ausführungen beschreiben: Das erste Lemma behandelt den Startfall: Liegen keine Events vor, so ist die Menge der gesendeten Nachrichten leer und die aktuellen Zustände entsprechen genau den Startzuständen. Das zweite Lemma beschreibt den induktiven Schritt: Jede valide Ausführung mit mindestens einem Event lässt sich auf eine kürzere valide Ausführung zurückführen. Das dritte Lemma behandelt Receive-Ereignisse: Tritt ein solches Event auf, so wurde die entsprechende Nachricht zuvor tatsächlich gesendet und muss sich in der Menge der gesendeten Nachrichten befinden [5].

4.2 Übergangsfunktionen für das Zwei-Phasen-Commit-Protokoll

Die Übergangsfunktion, die für das Zwei-Phasen-Commit-Protokoll benutzt wird sieht wie folgt aus:

```
47 fun tupac_step where
48   <tupac_step proc = (if proc = 0 then coordinator_step proc else
    ↪ participant_step)>
```

Hier wird die Annahme getroffen, dass Prozess 0 der Koordinator ist. Die Funktionen `coordinator_step` und `participant_step` stellen die spezifische Übergangsfunktionen für Koordinator und Teilnehmer dar und sind wie folgt definiert:

```
14 fun coordinator_step::
15   "'proc ⇒ ('t, 'proc) state ⇒ ('proc, 't msg) event ⇒ ('t, 'proc)
    ↪ state × ('proc, 't msg) send set" where
16   <coordinator_step pid (Initial t a r) Start = (Collecting t a {pid} r,
    ↪ {Send p (Prepare t) | p. p ∈ a})> |
17   <coordinator_step pid (Collecting t a y r) (Receive sender msg) =
18     (case msg of
19       Yes ⇒ (if y ∪ {sender} = a then (Committed a {pid}, {Send p
    ↪ (Commit) | p. p ∈ a}) else (Collecting t a (y ∪ {sender})
    ↪ r, {})) |
20       No ⇒ (Aborted a {pid}, {Send p (Abort) | p. p ∈ a}) |
21       _ ⇒ (Collecting t a y r, {}))> |
22   <coordinator_step pid (Collecting t a y 0) Timeout = (Aborted a {pid},
    ↪ {Send p (Abort) | p. p ∈ a})> |
23   <coordinator_step _ (Collecting t a y (Suc r)) Timeout = (Collecting t
    ↪ a y r, {Send p (Prepare t) | p. p ∈ (a - y)})> |
24   <coordinator_step _ (Committed a ack') (Receive sender Ack) =
25     (if {sender} ∪ ack' = a then (Forgotten, {}) else (Committed a
    ↪ (ack' ∪ {sender}), {}))> |
26   <coordinator_step _ (Committed a ack') Timeout = (Committed a ack',
    ↪ {Send p Commit | p. p ∈ (a - ack')})> |
27   <coordinator_step _ (Committed a ack') Restart = (Committed a ack',
    ↪ {Send p Commit | p. p ∈ a})> |
28   <coordinator_step _ (Aborted a ack') (Receive sender Ack) =
29     (if {sender} ∪ ack' = a then (Forgotten, {}) else (Aborted a (ack'
    ↪ ∪ {sender}), {}))> |
30   <coordinator_step _ (Aborted a ack') Timeout = (Aborted a ack', {Send
    ↪ p Abort | p. p ∈ (a - ack')})> |
31   <coordinator_step _ (Aborted a ack') Restart = (Aborted a ack', {Send
    ↪ p Abort | p. p ∈ a})> |
32   <coordinator_step _ state _ = (state, {})>

34 fun participant_step::
35   "'('t, 'proc) state ⇒ ('proc, 't msg) event ⇒ ('t, 'proc) state ×
    ↪ ('proc, 't msg) send set" where
```

```

36  <participant_step (Initial t _ _) (Receive sender (Prepare t')) = (if
    ↪  t = t' then (Prepared, {Send sender Yes}) else (Aborted {} {}),
    ↪  {Send sender No}))> |
37  <participant_step (Initial _ _ _) Timeout = (Aborted {} {}, {})> |
38  <participant_step Prepared (Receive sender msg) =
39      (case msg of
40          Commit ⇒ (Committed {} {}, {Send sender Ack}) |
41          Abort ⇒ (Aborted {} {}, {Send sender Ack}) |
42          _ ⇒ (Prepared, {}))> |
43  <participant_step (Committed _ _) (Receive sender Commit) = (Committed
    ↪  {} {}, {Send sender Ack})> |
44  <participant_step (Aborted _ _) (Receive sender Abort) = (Aborted {}
    ↪  {}, {Send sender Ack})> |
45  <participant_step state _ = (state, {})>

```

Diese Funktionen stellen das bereits beschriebene Verhalten von Koordinator und Teilnehmern formal dar, wobei die Zustände und Nachrichten als eigener Datentypen definiert werden:

```

5  datatype 't msg = Prepare (transaction: 't) | Yes | No | Commit | Abort
    ↪  | Ack
6
7  datatype ('t, 'proc) state = Initial (transaction: 't) (all: "'proc
    ↪  set") (retry_count:"nat")
8  | Collecting (transaction: 't) (all: "'proc set") (yes:"'proc set")
    ↪  (retry_count:"nat")
9  | Prepared
10 | Committed (all: "'proc set") (ack: "'proc set")
11 | Aborted (all: "'proc set") (ack: "'proc set")
12 | Forgotten

```

Die Zustände entsprechen jeweils den Zuständen aus der Protokollspezifikation.

4.3 Invarianten

Um die Eigenschaft der einheitlichen Zustimmung zu zeigen werden vorher Invarianten bewiesen, welche bei jeder validen Ausführung des Zwei-Phasen-Protokolls halten müssen [5]. Konkret werden in diesem Fall folgende neun Invarianten benötigt:

Invariante 1 besagt, dass wenn ein Teilnehmer im Committed-Zustand ist, dass eine Commit-Nachricht gesendet wurde.

```

8  definition inv1 where
9      <inv1 msgs states ↔
10          ((∃proc a ack'. proc ≠ 0 ∧ states proc = Committed a ack') →
11              (∃sender rcpt. (sender, Send rcpt Commit) ∈ msgs))>

```

Invariante 11 besagt, dass wenn es eine Commit-Nachricht gibt oder der Koordinator im Committed-Zustand ist, dass dann jeder Teilnehmer eine Yes-Nachricht gesendet haben muss.

```

13 definition inv11 where
14   <inv11 msgs states procs  $\longleftrightarrow$ 
15     (( $\exists$ sender rcpt. (sender, Send rcpt Commit)  $\in$  msgs)  $\vee$  ( $\exists$ all ack.
16        $\hookrightarrow$  states  $\emptyset$  = Committed all ack)  $\longrightarrow$ 
17         ( $\forall p \in$  procs.  $p \neq \emptyset \longrightarrow (\exists$ rcpt. ( $p$ , Send rcpt Yes)  $\in$ 
18           msgs)))>

```

Invariante 13 besagt, dass wenn der Koordinator im Collecting-Zustand ist, dass dann jeder Teilnehmer von dem er sich gespeichert hat eine Yes-Nachricht bekommen zu haben, auch wirklich eine Yes-Nachricht gesendet hat und dass die vom Koordinator gespeicherte Menge an teilnehmenden Prozessen auch der tatsächlichen Menge an teilnehmenden Prozessen entspricht.

```

22 definition inv13 where
23   <inv13 msgs states procs  $\longleftrightarrow$ 
24     ( $\forall$ t all yes r. states  $\emptyset$  = Collecting t all yes r  $\longrightarrow$ 
25       (procs = all  $\wedge$  ( $\forall p \in$  yes.  $p \neq \emptyset \longrightarrow (\exists$ rcpt. ( $p$ , Send rcpt
26         Yes)  $\in$  msgs))))>

```

Invariante 14 besagt, dass wenn sich der Koordinator im Initial-Zustand befindet, dass die von ihm gespeicherte Menge an teilnehmenden Prozessen auch der tatsächlichen Menge an teilnehmenden Prozessen entspricht.

```

27 definition inv14 where
28   <inv14 msgs states procs  $\longleftrightarrow$ 
29     ( $\forall$ t all r. states  $\emptyset$  = Initial t all r  $\longrightarrow$  procs = all)>

```

Invariante 15 besagt, dass wenn sich der Koordinator im Initial-, Collecting- oder Committed-Zustand befindet, noch keine Abort-Nachricht gesendet wurde.

```

31 definition inv15 where
32   <inv15 msgs states  $\longleftrightarrow$ 
33     (( $\exists$ t all yes r. states  $\emptyset$  = Initial t all r  $\vee$  states  $\emptyset$  = Collecting t
34        $\hookrightarrow$  all yes r  $\vee$  states  $\emptyset$  = Committed all yes)  $\longrightarrow$ 
35       ( $\forall$ sender rcpt. (sender, Send rcpt Abort)  $\notin$  msgs)))>

```

Invariante 17 besagt, dass wenn sich ein Teilnehmer im Aborted-Zustand befindet und keine Abort-Nachricht gesendet wurde, dass dann dieser Teilnehmer keine Yes-Nachricht geschickt hat.

```

40 definition inv17 where
41   <inv17 msgs states  $\longleftrightarrow$ 
42     ( $\forall p$ .  $p \neq \emptyset \wedge (\exists$ all ack. states  $p$  = Aborted all ack)  $\wedge$  ( $\forall$ sender
43       rcpt. (sender, Send rcpt Abort)  $\notin$  msgs)  $\longrightarrow$  ( $\forall$ rcpt. ( $p$ , Send
44       rcpt Yes)  $\notin$  msgs)))>

```

Invariante 110 besagt, dass wenn sich der Koordinator im Initial-, Collecting- oder Aborted-Zustand befindet, noch keine Commit-Nachricht gesendet wurde.

```
52 definition inv110 where
53   <inv110 msgs states  $\longleftrightarrow$ 
54     (( $\exists t$  all yes r. states  $\emptyset$  = Initial t all r  $\vee$  states  $\emptyset$  = Collecting t
55        $\hookrightarrow$  all yes r  $\vee$  states  $\emptyset$  = Aborted all yes)  $\longrightarrow$ 
56       ( $\forall$ sender rcpt. (sender, Send rcpt Commit)  $\notin$  msgs))>
```

Invariante 111 besagt, dass wenn eine Commit-Nachricht gesendet wurde, sich der Koordinator entweder im Committed- oder Forgotten-Zustand befindet.

```
57 definition inv111 where
58   <inv111 msgs states  $\longleftrightarrow$ 
59     (( $\exists$ sender rcpt. (sender, Send rcpt Commit)  $\in$  msgs)  $\longrightarrow$  ( $\exists$ all ack.
60        $\hookrightarrow$  states  $\emptyset$  = Committed all ack  $\vee$  states  $\emptyset$  = Forgotten))>
```

Invariante 3 besagt, dass wenn eine Commit-Nachricht gesendet wurde, keine Abort-Nachricht gesendet wurde.

```
67 definition inv3 where
68   <inv3 msgs states  $\longleftrightarrow$ 
69     ( $\exists$ sender rcpt. (sender, Send rcpt Commit)  $\in$  msgs)  $\longrightarrow$  ( $\forall$ sender
70        $\hookrightarrow$  rcpt. (sender, Send rcpt Abort)  $\notin$  msgs)>
```

4.4 Beweis der Invarianten

Diese Invarianten müssen nun für jede gültige Ausführung des zwei-Phase-Protokolls bewiesen werden. Da sich die Beweise der verschiedenen Invarianten nicht wesentlich unterscheiden, wird hier beispielhaft der Beweis für Invariante 1 gezeigt.

Der Beweis nimmt eine valide Ausführung mit der Übergangsfunktion des Zwei-Phasen-Commit-Protokolls an, bei der alle Prozesse im Initial-Zustand starten und zeigt damit, dass die Invariante 1 für die in dieser Ausführung gesendeten Nachrichten und aktuelle Zustandsfunktion gilt:

```
90 lemma invariant1:
91   assumes <execute tupac_step ( $\lambda p$ . Initial (init_val p) UNIV r) UNIV
92      $\hookrightarrow$  events msgs' states'>
93   shows "inv1 msgs' states'"
```

Der Beweis wird als Induktion über die Liste der Events geführt, wobei der Basisfall, die leere Liste, mittels eines Hilfslemmas des Frameworks einfach bewiesen werden kann:

```
93 using assms proof(induction events arbitrary: msgs' states' r rule:
94    $\hookrightarrow$  List.rev_induct)
95 case Nil
96 then have statesInit:" $\forall p$ . states' p = Initial (init_val p) UNIV r"
```

```

96   using execute_init assms by fast
97   moreover have msgsEmpty: "msgs' = {}"
98   using execute_init Nil by fast
99   ultimately show ?case
100   using inv1_def by auto

```

Im Induktionsschritt wird angenommen, dass die Invariante für eine gewisse Liste an Events gilt. Nun muss gezeigt werden, dass das ausführen eines weiteren Events, die Gültigkeit der Invariante erhält. Um dies zu zeigen wird zuerst das neu angehängte Listenelement in das eigentliche Event und den Prozess der dieses Event verarbeitet aufgeteilt:

```

102   case (snoc x events)
103   obtain proc event where "x = (proc, event)"
104   by fastforce

```

Darauf aufbauend, werden Hilfseigenschaften für später gezeigt:

```

105   hence exec: "execute tupac_step (λp. Initial (init_val p) UNIV r) UNIV
106               (events @ [(proc, event)]) msgs' states'"
107   using snoc.premss assms by fast
108   from this obtain msgs states sent new_state
109   where step_rel1: "execute tupac_step (λp. Initial (init_val p) UNIV
110     ↪ r) UNIV events msgs states"
111     and step_rel2: "tupac_step proc (states proc) event = (new_state,
112     ↪ sent)"
113     and step_rel3: "msgs' = msgs ∪ ((λmsg. (proc, msg)) ` sent)"
114     and step_rel4: "states' = states (proc := new_state)"
115   by auto

```

Anschließend wird eine Fallunterscheidung gemacht je nachdem ob es sich bei dem ausführenden Prozess um den Koordinator oder einen Teilnehmer handelt. Innerhalb dieser Fallunterscheidungen kommt nun jeweils ein Hilfslemma zum Einsatz, mit dessen Hilfe der Beweis in beiden Fällen abgeschlossen werden kann:

```

114   show ?case
115   proof (cases "proc = 0")
116     case True
117     then have "coordinator_step proc (states 0) event = (new_state,
118     ↪ sent)"
119     using step_rel2 by simp
120     moreover have "inv1 msgs states"
121     using snoc.IH step_rel1 by fast
122     ultimately show ?thesis
123     using invariant1_coordinator True step_rel3 step_rel4 exec by metis
124   next
125     case False
126     then have "participant_step (states proc) event = (new_state, sent)"
127     using step_rel2 by simp

```



```

127   moreover have "inv1 msgs states"
128   using snoc.IH step_rel1 by fast
129   ultimately show ?thesis
130   using invariant1_participant False step_rel3 step_rel4 exec by
      ↪ metis
131 qed
132 qed

```

4.4.1 Hilfslemma für Koordinator

Das Hilfslemma für den Fall, dass es sich um den Koordinator handelt nimmt eine valide Ausführung der Übergangsfunktion des Koordinators, eine bestimmte Zusammensetzung von Nachrichten und Events und die Gültigkeit der Invariante im vorherigen Schritt an:

```

5 lemma invariant1_coordinator:
6   assumes "coordinator_step 0 (states 0) event = (new_state, sent)"
7   and "msgs' = msgs ∪ ((λmsg. (0, msg)) ` sent)"
8   and "states' = states (0 := new_state)"
9   and "execute tupac_step (λp. Initial (init_val p) UNIV r) UNIV
      ↪ (events @ [(0, event)]) msgs' states'"
10  and "inv1 msgs states"

```

Um zu zeigen, dass Invariante 1 auch nach dem Übergang des Koordinators gilt, wird zuerst eine Fallunterscheidung über das neue Event, den aktuellen Zustand des Koordinators und im Falle eines Receive-Events auch über die eingehende Nachricht gemacht. Dabei entstehen jedoch viele Fälle, in denen sich weder der Zustand des Koordinators ändert noch irgendwelche Nachrichten gesendet werden. Solche Fälle werden vom Hilfslemma `coordinator_induct` übernommen, sodass nur der Beweis der nicht trivialen Fälle bleibt, welche einfach bewiesen werden können:

```

11 shows "inv1 msgs' states'"
12 using assms apply (induction rule: coordinator_induct)
13 using inv1_def UnCI assms(2,3,5) apply (metis (mono_tags, lifting)
      ↪ fun_upd_apply)+
14 done

```

4.4.2 Hilfslemma für Teilnehmer

Das Hilfslemma für den Fall, dass es sich um einen Teilnehmer handelt, nimmt bis auf eine valide Ausführung der Übergangsfunktion des Teilnehmers, die gleichen Argumente wie das Hilfslemma für den Koordinator an:

```

16 lemma invariant1_participant:
17   assumes "participant_step (states proc) event = (new_state, sent)"
18   and "proc ≠ 0"
19   and "msgs' = msgs ∪ ((λmsg. (proc, msg)) ` sent)"

```

```

20   and "states' = states (proc := new_state)"
21   and "execute tupac_step ( $\lambda$ p. Initial (init_val p) UNIV r) UNIV
     $\hookrightarrow$  (events @ [(proc, event)]) msgs' states'"
22   and "inv1 msgs states"

```

Um dieses Lemma zu beweisen, wird genau wie beim Koordinator zuerst eine Fallunterscheidung über das neue Event, den aktuellen Zustand des Teilnehmers und im Falle eines Receive-Events auch über die eingehende Nachricht gemacht. Die trivialen Fälle werden hierbei vom Hilfslemma `participant_induct` übernommen:

```

23   shows "inv1 msgs' states'"
24   using assms
25   proof (induction rule: participant_induct)

```

Für die restlichen Fälle wird jeweils eine Beispielbeweissführung gezeigt.

Sollte der Teilnehmer von einem nicht Committed-Zustand zu einem nicht Committed-Zustand übergehen, so befindet sich der Koordinator im Comitted-Zustand genau dann, wenn er sich vorher bereits dort befand:

```

26   case (Init_Prep_Yes t a r sender t')
27   then have " $\exists$ proc a ack'. (proc  $\neq$  0  $\wedge$  states proc = Committed a ack')
     $\hookrightarrow$   $\longleftrightarrow$  (proc  $\neq$  0  $\wedge$  states' proc = Committed a ack')"
28   using assms(4) by metis

```

Darüber hinaus wurde eine Commit-Nachricht genau dann gesendet, wenn sie vorher bereits gesendet wurde, da der Teilnehmer eine solche Nachricht nicht senden kann:

```

29   moreover have " $\exists$ sender rcpt. ((sender, Send rcpt Commit)  $\in$  msgs)  $\longleftrightarrow$ 
     $\hookrightarrow$  ((sender, Send rcpt Commit)  $\in$  msgs)'"
30   using assms(3) Init_Prep_Yes(5) by simp

```

Daraus folgt, dass Invariante 1 unverändert bleibt. Da sie vorher galt, gilt sie nun auch nach verarbeiten des neuen Events.

```

31   ultimately show ?case
32   by (metis (no_types, lifting) Init_Prep_Yes(4) UnCI assms(3,4,6)
     $\hookrightarrow$  fun_upd_apply inv1_def
33   state.distinct(19))

```

Sollte der Teilnehmer von einem nicht Committed-Zustand zu einem Committed-Zustand übergehen, so tut er dies nur nach Erhalt einer Commit-Nachricht, mit dem Hilfslemma `execute_receive` ergibt sich, dass eine solche Nachricht gesendet wurde:

```

53   case (Prep_Commit sender)
54   then have " $\exists$ sender rcpt. (sender, Send rcpt Commit)  $\in$  msgs"
55   using assms(3,5) execute_receive Un_commute image_insert
     $\hookrightarrow$  image_is_empty by (smt (verit, ccfv_SIG)
56   insert_iff insert_is_Un msg.distinct(27) prod.inject
     $\hookrightarrow$  send.inject)

```

Dies gilt auch nach dem Übergang des Teilnehmers, woraus direkt die Gültigkeit der Invariante 1 folgt:

```

57   then have "∃sender rcpt. (sender, Send rcpt Commit) ∈ msgs'"
58   using assms(3) by auto
59   then show ?case
60   using inv1_def by blast

```

Bleibt der Teilnehmer in einem Committed-Zustand, so wurde nach Invariante 1 bereits eine Commit-Nachricht gesendet:

```

71   case (Committed_Commit all ack sender)
72   then have "∃sender rcpt. (sender, Send rcpt Commit) ∈ msgs"
73   using assms(3,5) execute_receive Un_commute image_insert
    ↪ image_is_empty by (smt (verit, ccfv_SIG)
74       insert_iff insert_is_Un msg.distinct(27) prod.inject
    ↪ send.inject)

```

Dies gilt ebenfalls nach dem Übergang des Teilnehmers, woraus erneut direkt die Gültigkeit der Invariante 1 folgt:

```

75   then have "∃sender rcpt. (sender, Send rcpt Commit) ∈ msgs'"
76   using assms(3) by auto
77   then show ?case
78   using inv1_def by blast

```

Ein Teilnehmer kann laut seiner Übergangsfunktion von einem Committed-Zustand nicht in einen nicht Committed-Zustand gelangen, weshalb dieser Fall entfällt.

4.5 Beweis der einheitlichen Zustimmung

Da nun alle nötigen Invarianten bewiesen wurden, kann nun die Eigenschaft der einheitlichen Zustimmung bewiesen werden. Das lemma dafür nimmt eine valide Ausführung des Zwei-Phasen-Commit-Protokolls an, sowie zwei (nicht notwendigerweise verschiedene) Prozesse, die sich jeweils im Committed- oder Aborted-Zustand befinden. Daraus wird gezeigt, dass sich ein Prozess genau dann im Committed-Zustand befindet, wenn sich auch der andere Prozess im Committed-Zustand befindet, was genau der Eigenschaft der einheitlichen Zustimmung entspricht:

```

98   theorem ac1 :
99   assumes <execute tupac_step (λp. Initial (init_val p) UNIV r) UNIV
    ↪ events msgs states>
100   and <states proc1 = Committed a b ∨ states proc1 = Aborted a b>
101   and <states proc2 = Committed c d ∨ states proc2 = Aborted c d>
102   shows <states proc1 = Committed a b ↔ states proc2 = Committed c d>

```

Zuerst wird eine Fallunterscheidung durchgeführt, je nachdem ob es sich bei den Prozessen um den gleichen Prozess handelt. Ist dies der Fall ist der Beweis trivial:

```

103 proof(cases "proc1 = proc2")
104   case True
105   then show ?thesis
106   using assms by auto

```

Sollte dies nicht der Fall sein, wird eine weitere Fallunterscheidung durchgeführt, ob es sich bei einem der Prozess um den Koordinator handelt. Ist dies der Fall, so kann der Beweis mit Hilfe des Lemmas `ac1_OneCoordinator` beendet werden:

```

108   case False
109   then show ?thesis
110   proof(cases "proc1 = 0 ∨ proc2 = 0")
111     case True
112     then show ?thesis
113     using ac1_OneCoordinator by (metis assms(1-3) state.distinct(25))

```

Andernfalls hilft das Lemma `ac1_NoCoordinator` beim Beweis:

```

115   case False
116   then have "proc1 ≠ 0 ∧ proc2 ≠ 0"
117   by satx
118   then show ?thesis
119   using ac1_NoCoordinator assms by blast

```

Der Beweis für das Hilfslemma `ac1_OneCoordinator` trifft die gleichen Annahmen wie das allgemeine Lemma `ac1`. Jedoch nimmt es zusätzlich an, dass genau ein Prozess der Koordinator ist:

```

58 theorem ac1_OneCoordinator:
59   assumes <execute tupac_step (λp. Initial (init_val p) UNIV r) UNIV
        ⇨ events msgs states>
60   and <p ≠ 0>
61   and <states 0 = Committed a b ∨ states 0 = Aborted a b>
62   and <states p = Committed c d ∨ states p = Aborted c d>
63   shows <states 0 = Committed a b ⟷ states p = Committed c d>

```

Der Beweis wird in erster Linie wie ein normaler Äquivalenzbeweis geführt. Für die Hinrichtung, sollte sich der Koordinator im Committed-Zustand befinden, zeigt ein anschließender Beweis durch Widerspruch, dass sich der Teilnehmer auch im Committed-Zustand befinden muss:

```

64 proof
65   assume asmCommitted:"states 0 = Committed a b"
66   have inv:"inv11 msgs states UNIV ∧ inv17 msgs states ∧ inv15 msgs
        ⇨ states"
67   using invariants123 invariant15 assms(1) by blast
68   show "states p = Committed c d"
69   proof(rule ccontr)

```

Zuerst wird angenommen, dass sich der Teilnehmer nicht im Committed-Zustand befindet, weshalb er sich laut Annahmen im Aborted-Zustand befinden muss:

```
70   assume asmAborted: "states p ≠ Committed c d"
71   then have proc2Abort: "states p = Aborted c d"
72   using assms(4) by auto
```

Zusätzlich muss laut Invariante 11 jeder Teilnehmer eine Yes-Nachricht gesendet haben, da sich der Koordinator per Annahme im Committed-Zustand befindet:

```
73   then have "(∀p ∈ UNIV. p ≠ 0 → (∃rcpt. (p, Send rcpt Yes) ∈ msgs))"
74   using inv inv11_def by (metis asmCommitted)
```

Der betrachtete Teilnehmer, der sich im Aborted-Zustand befindet, muss also ebenfalls eine Yes-Nachricht gesendet haben:

```
75   then have "∃rcpt. (p, Send rcpt Yes) ∈ msgs"
76   using assms(2) by auto
```

Darüber hinaus kann keine Abort-Nachricht gesendet worden sein, da sich der Koordinator im Committed-Zustand befindet:

```
77   moreover have noAbort: "∀sender rcpt. (sender, Send rcpt Abort) ∉
    ↪ msgs"
78   using inv15_def asmCommitted inv by metis
```

Daraus folgt samt Invariante 17, dass der Teilnehmer im Abort-Zustand keine Yes-Nachricht geschickt haben kann, was einen Widerspruch ergibt.

```
79   moreover have "∀rcpt. (p, Send rcpt Yes) ∉ msgs"
80   using proc2Abort assms(2) noAbort by (meson inv inv17_def)
81   ultimately show "False"
82   by simp
```

Die Rückrichtung, sollte sich der Teilnehmer im Committed-Zustand befinden, wird als direkter Beweis geführt. Da sich der Teilnehmer im Committed-Zustand befindet, muss eine Commit-Nachricht gesendet worden sein:

```
86   assume asmCommitted: "states p = Committed c d"
87   have inv: "inv1 msgs states ∧ inv111 msgs states"
88   using invariants123 invariant1 invariant111 assms(1) invariant3 by
    ↪ blast
89   have commitMsg: "(∃sender rcpt. (sender, Send rcpt Commit) ∈ msgs)"
90   using inv asmCommitted inv1_def assms(2) by metis
```

Daraus folgt, dass sich der Koordinator entweder im Committed- oder Forgotten-Zustand befindet, wodurch zusammen mit den Annahmen des Lemmas der direkte Beweis beendet werden kann.

```

91  then have "∃all ack. states 0 = Committed all ack ∨ states 0 =
    ↪  Forgotten"
92  using inv inv111_def by metis
93  then show "states 0 = Committed a b"
94  using assms(3) by force

```

Der Beweis für das Hilfslemma `ac1_NoCoordinator` trifft die gleichen Annahmen wie das allgemeine Lemma `ac1`. Jedoch nimmt es zusätzlich an, dass kein Prozess der Koordinator ist:

```

10  theorem ac1_NoCoordinator:
11  assumes <execute tupac_step (λp. Initial (init_val p) UNIV r) UNIV
    ↪  events msgs states>
12  and <proc1 ≠ 0 ∧ proc2 ≠ 0>
13  and <states proc1 = Committed a b ∨ states proc1 = Aborted a b>
14  and <states proc2 = Committed c d ∨ states proc2 = Aborted c d>
15  shows <states proc1 = Committed a b ↔ states proc2 = Committed c d>

```

Der Beweis wird als Widerspruchsbeweis geführt. Es wird angenommen, dass die Äquivalenz der Zustände nicht gilt:

```

16  proof (rule ccontr)
17  assume asm1:"¬(states proc1 = Committed a b ↔ states proc2 =
    ↪  Committed c d)"
18  have inv:"inv1 msgs states ∧ inv3 msgs states ∧ inv11 msgs states UNIV
    ↪  ∧ inv17 msgs states"
19  using invariants123 invariant1 invariant3 assms(1) by blast

```

Daraufhin wird unterschieden ob sich der erste Teilnehmer im Committed-Zustand befindet. Beide Fälle können analog bewiesen werden, deswegen wird hier nur der Fall dafür gezeigt, dass sich der erste Teilnehmer im Committed-Zustand befindet.

Da die Äquivalenz der Zustände per Annahme nicht gilt, muss sich der andere Teilnehmer im Aborted-Zustand befinden:

```

20  then show "False"
21  proof(cases "states proc1 = Committed a b")
22  case True
23  then have proc2Abort:"states proc2 = Aborted c d"
24  using asm1 assms(4) by auto

```

Weiterhin muss laut Invariante 1, aufgrund des Zustandes des ersten Teilnehmers, eine Commit-Nachricht gesendet worden sein:

```

25  then have commitMsg:"(∃sender rcpt. (sender, Send rcpt Commit) ∈
    ↪  msgs)"
26  using True inv by (metis assms(2) inv1_def)

```

Zusammen mit Invariante 3 folgt daraus, dass keine Abort-Nachricht gesendet werden konnte:

```

27   then have noAbort: "( $\forall$ sender rcpt. (sender, Send rcpt Abort)  $\notin$ 
     $\hookrightarrow$  msgs)"
28   using inv inv3_def by metis

```

Da eine Commit-Nachricht gesendet wurde, muss nach Invariante 11 jeder Teilnehmer, also auch der zweite Teilnehmer, eine Yes-Nachricht gesendet haben:

```

29   then have "( $\forall p \in \text{UNIV}. p \neq 0 \longrightarrow (\exists \text{rcpt}. (p, \text{Send rcpt Yes}) \in \text{msgs}))"$ 
30   using inv inv11_def by (metis commitMsg)
31   then have " $\exists \text{rcpt}. (\text{proc2}, \text{Send rcpt Yes}) \in \text{msgs}"$ 
32   using assms(2) by auto

```

Da sich dieser aber im Aborted-Zustand befindet und keine Abort-Nachrichten gesendet wurden, hat er nach Invariante 17 keine Yes-Nachricht gesendet, was zum Widerspruch führt:

```

33   moreover have " $\forall \text{rcpt}. (\text{proc2}, \text{Send rcpt Yes}) \notin \text{msgs}"$ 
34   using proc2Abort assms(2) noAbort by (meson inv inv17_def)
35   ultimately show ?thesis
36   by simp

```

Somit konnte mittels des Modells von Kleppmann die Eigenschaft der einheitlichen Zustimmung, für das Zwei-Phasen-Commit-Protokolls, auch mit eventuell verloren gehenden Nachrichten, duplizierten Nachrichten oder ausfallenden Prozessen bewiesen werden.

5 I/O Automaten

Dieses Kapitel gibt eine kurze Einführung zu den von Nancy A. Lynch und Mark R. Tuttle eingeführten Input/Output Automaten (IOA) [11]. Weiterhin wird das Zwei-Phasen-commit-Protocol in Isabelle mittels IOA modelliert und das zugehörige Framework vorgestellt. Anschließend wird mittels Invarianten die Eigenschaft der einheitlichen Zustimmung für das Protokoll bewiesen.

5.1 Mathematische Beschreibung nach Lynch

Ein IOA ist ein Modell zur Beschreibung von Komponenten eines verteilten Systems und dessen Interaktionen mit der Umgebung. Ein solcher IOA kann als Automat gesehen werden, dessen Übergänge jeweils als Input, Output oder Internal klassifiziert werden können. Input- und Output-Übergänge ermöglichen dabei die Kommunikation mit der Umgebung und anderen IOA während Internal-Übergänge nur lokal von diesem Automaten benutzt werden [6].

Eine wichtige Eigenschaft dabei ist, dass Input-Übergänge nicht vom Automaten selber kontrolliert werden, sondern jederzeit auftreten können. Das führt dazu, dass IOA in jedem Zustand in der Lage sein müssen, jegliche Input-Übergänge verarbeiten zu können [6].

Formal ist ein IOA ein fünf-Tupel bestehend aus [6]:

- eine Signatur, welche als Tripel aus den 3 disjunkten Übergangsmengen besteht
- eine Menge an Zustände
- eine nichtleere Menge an Startzuständen
- eine Übergangsrelation, welche definiert von welchen Zuständen und mit welchen Übergängen der Automat in welchen Zuständen landen kann
- eine Aufgabenpartition, welche zur Modellierung von Fairness-Bedingungen benutzt werden kann

5.1.1 Ausführung eines IOA

Formal wird ein Ausführungsfragment eines IOA als (unendliche) Folge $s_0, \pi_1, s_1, \pi_2, \dots$ bestehend abwechselnd aus Zustand und Übergang beschrieben. Dabei muss jedes Tripel $(s_i, \pi(i+1), s(i+1))$ in der Übergangsrelation enthalten sein.

Ein Ausführungsfragment, welches mit einem Startzustand anfängt, heißt Ausführung. Ein Zustand heißt erreichbar für einen IOA, wenn er als letzter Zustand in einer endlichen Ausführung dieses IOA vorkommt.

5.1.2 Komposition

Mehrere IOA können durch Komposition zu einem IOA kombiniert werden. Dies erlaubt es ein komplexes System durch seine Bestandteile zu modellieren. Formal lassen sich auch abzählbar unendlich viele IOA kombinieren, hier wird jedoch nur die Komposition von zwei IOA betrachtet [6].

Zwei IOA lassen sich kombinieren, wenn die Menge der Internal-Übergänge des einen Automaten disjunkt zur allen Übergangsmengen des anderen Automaten ist (und umgekehrt) und zudem die Mengen der Output-Übergänge beider Automaten paarweise disjunkt sind [6].

Bei einer solchen Komposition ist die Menge der Output-Übergänge des entstehenden IOA gleich der Vereinigung der Output-Übergänge beider Ausgangsautomaten. Analog entsteht die Menge der Internal-Übergänge. Bei der Menge der Input-Übergänge müssen nach der Vereinigung noch alle Output-Übergänge der ursprünglichen IOA abgezogen werden, dies sorgt dafür, dass nur solche Input-Übergänge übrig bleiben, zu denen der andere IOA keine Output-Übergänge besitzt [6].

5.2 I/O Automaten in Isabelle

Das hier vorgestellte Isabelle Framework für IOA kommt mit Isabelle mitgeliefert und wurde von Olaf Müller, Konrad Slind und Tobias Nipkow geschrieben [12]. Es besteht aus zwei Dateien, von denen eine die Formalisierung der Signatur eines IOA übernimmt:

```

11 type_synonym 'a signature = "'a set × 'a set × 'a set"
12
13 definition inputs :: "'action signature ⇒ 'action set"
14   where asig_inputs_def: "inputs = fst"
15
16 definition outputs :: "'action signature ⇒ 'action set"
17   where asig_outputs_def: "outputs = fst ∘ snd"
18
19 definition internals :: "'action signature ⇒ 'action set"
20   where asig_internals_def: "internals = snd ∘ snd"

```

Neben diesen Definitionen enthält diese Datei auch eine Definition wann eine Signatur gültig ist, sowie weitere kleine Hilfslemmas bezüglich Signaturen [12]. Die Definitionen sind dabei zumeist stark dem mathematischen Modell ähnelnd [6].

Die andere Datei des Frameworks führt die IOA mithilfe dieser Signaturen ein:

```

13 type_synonym ('a, 's) transition = "'s × 'a × 's"
14 type_synonym ('a, 's) ioa =
15   "'a signature × 's set × ('a, 's)transition set × 'a set set × 'a set
    ⇨ set"
20 definition asig_of :: "('a, 's) ioa ⇒ 'a signature"
21   where "asig_of = fst"

```

```

22
23 definition starts_of :: "('a, 's) ioa  $\Rightarrow$  's set"
24   where "starts_of = fst  $\circ$  snd"
25
26 definition trans_of :: "('a, 's) ioa  $\Rightarrow$  ('a, 's) transition set"
27   where "trans_of = fst  $\circ$  snd  $\circ$  snd"
28
29 abbreviation trans_of_syn ( $\langle\_ \_ \_ \_ \rangle$  [81, 81, 81, 81] 100)
30   where "s -a-A $\rightarrow$  t  $\equiv$  (s, a, t)  $\in$  trans_of A"
31
32 definition wfair_of :: "('a, 's) ioa  $\Rightarrow$  'a set set"
33   where "wfair_of = fst  $\circ$  snd  $\circ$  snd  $\circ$  snd"
34
35 definition sfair_of :: "('a, 's) ioa  $\Rightarrow$  'a set set"
36   where "sfair_of = snd  $\circ$  snd  $\circ$  snd  $\circ$  snd"

```

Dabei wird die Menge aller Zustände nicht explizit mit angegeben. Die Mengen `wfair_of` und `sfair_of` werden zur Beschreibung von Fairness benutzt und hier nicht weiter betrachtet.

Weiterhin wird auch der Kompositionsoperator `par ||` für zwei IOA eingeführt:

```

88 definition asig_comp :: "'a signature  $\Rightarrow$  'a signature  $\Rightarrow$  'a signature"
89   where "asig_comp a1 a2 =
90     (((inputs a1  $\cup$  inputs a2) - (outputs a1  $\cup$  outputs a2),
91      (outputs a1  $\cup$  outputs a2),
92      (internals a1  $\cup$  internals a2)))"
93
94 definition par :: "('a, 's) ioa  $\Rightarrow$  ('a, 't) ioa  $\Rightarrow$  ('a, 's * 't) ioa"
95    $\hookrightarrow$  (infixr  $\langle || \rangle$  10)
96   where "(A || B) =
97     (asig_comp (asig_of A) (asig_of B),
98      {pr. fst pr  $\in$  starts_of A  $\wedge$  snd pr  $\in$  starts_of B},
99      {tr.
100        let
101          s = fst tr;
102          a = fst (snd tr);
103          t = snd (snd tr)
104        in
105          (a  $\in$  act A  $\vee$  a  $\in$  act B)  $\wedge$ 
106          (if a  $\in$  act A then (fst s, a, fst t)  $\in$  trans_of A
107           else fst t = fst s)  $\wedge$ 
108          (if a  $\in$  act B then (snd s, a, snd t)  $\in$  trans_of B
109           else snd t = snd s)},
110      wfair_of A  $\cup$  wfair_of B,
111      sfair_of A  $\cup$  sfair_of B)"

```

Ob ein Zustand erreichbar ist, wird wie folgt induktiv definiert:

```

69 inductive reachable :: "('a, 's) ioa  $\Rightarrow$  's  $\Rightarrow$  bool" for C :: "('a, 's)
    $\hookrightarrow$  ioa"
70 where
71   reachable_0: "s  $\in$  starts_of C  $\Rightarrow$  reachable C s"
72 | reachable_n: "reachable C s  $\Rightarrow$  (s, a, t)  $\in$  trans_of C  $\Rightarrow$  reachable
    $\hookrightarrow$  C t"

```

Um zu zeigen, dass eine Eigenschaft in jeder Ausführung wahr ist, hilft folgendes Lemma:

```

74 definition invariant :: "[('a, 's) ioa, 's  $\Rightarrow$  bool]  $\Rightarrow$  bool"
75   where "invariant A P  $\longleftrightarrow$  ( $\forall$ s. reachable A s  $\longrightarrow$  P s)"

380 lemma invariantI:
381   assumes " $\wedge$ s. s  $\in$  starts_of A  $\Rightarrow$  P s"
382   and " $\wedge$ s t a. reachable A s  $\Rightarrow$  P s  $\Rightarrow$  (s, a, t)  $\in$  trans_of A  $\longrightarrow$  P t"
383   shows "invariant A P"

```

Das Lemma zeigt, dass eine Eigenschaft invariant bezüglich Ausführungen ist. Dies wird gezeigt, indem die Eigenschaft in allen Startzuständen und nach jedem möglichen Übergang betrachtet wird.

Weiterhin stellt das Framework viele kleine Hilfslemmas zur Verfügung, um den Umgang mit den Definitionen zu erleichtern.

5.3 Übergangsrelationen für das Zwei-Phasen-Commit-Protokoll

Mithilfe des Frameworks kann nun mit der Implementierung des Zwei-Phasen-Commit-Protokolls begonnen werden. Der IOA für den Koordinator wird wie folgt definiert:

```

8 datatype ('proc, 'msg) action =
9   Start "'proc"
10  | Send "'proc" "'proc" "'msg"
11  | Receive "'proc" "'proc" "'msg"
12  | Timeout "'proc"
13  | Restart "'proc"

17 datatype ('t, 'proc) cstate = CInitial (transaction: 't) (all: "'proc
    $\hookrightarrow$  set") (retry_count:"nat")
18  | Collecting (transaction: 't) (all: "'proc set") (yes:"'proc set")
    $\hookrightarrow$  (retry_count:"nat")
19  | CCommitted (all: "'proc set") (ack: "'proc set")
20  | CAborted (all: "'proc set") (ack: "'proc set")
21  | Forgotten

22 datatype ('t, 'proc) coord_state = CState "('t, 'proc) cstate" "('proc  $\times$ 
    $\hookrightarrow$  'proc  $\times$  't msg) set"

```

Wobei der Datentyp der Nachrichten und die Funktion `coordinator_step` analog zum Modell von Kleppmann definiert werden. Letzteres wird aus Gründen der Übersichtlichkeit weggelassen.

Daraus wird die Menge aller Übergänge definiert, woraus zusammen mit der Signatur der IOA des Koordinators definiert werden kann:

```

44 definition coord_trans ::
45   "'proc  $\Rightarrow$  (('t, 'proc) coord_state  $\times$  ('proc, 't msg) action  $\times$  ('t, 'proc)
    $\hookrightarrow$  coord_state) set" where
46   "coord_trans pid =
47     {(CState s msgs, a, CState s' (msgs  $\cup$  (( $\lambda$ msg. (pid, msg))  $\sim$  sent))) |
    $\hookrightarrow$  s a s' msgs sent. coordinator_step pid s a = (s', sent)  $\wedge$  ( $\exists$ s m. a
    $\hookrightarrow$  = Start pid  $\vee$  a = Receive s pid m  $\vee$  a = Timeout pid  $\vee$  a = Restart
    $\hookrightarrow$  pid)}
48    $\cup$  {(CState s msgs, Send pid rcpt msg, CState s (msgs -
    $\hookrightarrow$  {(pid, rcpt, msg)})) | s msgs rcpt msg. (pid, rcpt, msg)  $\in$  msgs}"
49
50 definition coord_asig :: "'proc  $\Rightarrow$  ('proc, 't) action signature" where
51   "coord_asig pid =
52     ({Receive q pid m | q m. True}  $\cup$  {Start pid, Timeout pid, Restart pid},
53     {Send pid q m | q m. True},
54     {})"
55
56 definition Coordinator :: "'p  $\Rightarrow$  't  $\Rightarrow$  'p set  $\Rightarrow$  nat  $\Rightarrow$  (('p, 't msg)
    $\hookrightarrow$  action, ('t, 'p) coord_state) ioa" where
57   "Coordinator pid t a r = (coord_asig pid, {CState (CInitial t a r) {}},
    $\hookrightarrow$  coord_trans pid, {}, {})"

```

Anders als im Modell von Kleppmann, kann in dieser Modellierung immer nur eine Nachricht auf einmal gesendet werden. Deswegen werden alle zu sendenden Nachrichten in einem Extrazustand gesammelt und Nacheinander durch Send-Aktionen gesendet. Start-, Timeout- und Restart-Aktionen sind dabei als Input-Übergänge definiert, da sie so nicht unter die Kontrolle des IOA fallen und der Koordinator deswegen mit diesen Übergänge in jedem Zustand umgehen können muss.

Die Teilnehmer werden als ein IOA modelliert, um eine flexible Anzahl dieser darstellen zu können. Die Modellierung dieses IOA folgt analog der Modellierung des Koordinator IOA. Auch hier wird die Funktion `participant_step` analog zum Modell von Kleppmann definiert und weggelassen.

```

61 datatype 't pstate = PInitial (transaction: 't)
62   | Prepared
63   | PCommitted
64   | PAborted
65 datatype ('t, 'proc) part_state = PState "'t pstate" "('proc  $\times$  'proc  $\times$  't
    $\hookrightarrow$  msg) set"
66
81 definition part_trans ::

```

```

82   "'proc  $\Rightarrow$  (('t,'proc) part_state  $\times$  ('proc,'t msg) action  $\times$  ('t,'proc)
     $\hookrightarrow$  part_state) set" where
83 "part_trans pid =
84   {(PState s msgs, a, PState s' (msgs  $\cup$  (( $\lambda$ msg. (pid, msg))  $\setminus$  sent))) |
     $\hookrightarrow$  s a s' msgs sent. participant_step pid s a = (s', sent)  $\wedge$  ( $\exists$ s m. a
     $\hookrightarrow$  = Receive s pid m  $\vee$  a = Timeout pid  $\vee$  a = Restart pid)}
85  $\cup$  {(PState s msgs, Send pid rcpt msg, PState s (msgs -
     $\hookrightarrow$  {(pid,rcpt,msg)})) | s msgs rcpt msg. (pid,rcpt,msg)  $\in$  msgs}"

89 definition parts_asig :: "'proc set  $\Rightarrow$  ('proc,'t msg) action signature"
     $\hookrightarrow$  where
90 "parts_asig P =
91   ({Receive q p m | q p m. p  $\in$  P}  $\cup$  {Timeout p | p. p  $\in$  P}  $\cup$  {Restart p |
     $\hookrightarrow$  p. p  $\in$  P},
92   {Send p q m | p q m. p  $\in$  P},
93   {})"

94 definition parts_trans :: "'proc set  $\Rightarrow$  (('proc  $\Rightarrow$  ('t,'proc)
     $\hookrightarrow$  part_state)  $\times$  ('proc,'t msg) action  $\times$  ('proc  $\Rightarrow$  ('t,'proc)
     $\hookrightarrow$  part_state)) set" where
95 "parts_trans P = { (s, a, s').
96    $\forall$ p. if p  $\in$  P  $\wedge$  (( $\exists$ q m. a = Receive q p m)  $\vee$  a = Timeout p  $\vee$  a =
97    $\hookrightarrow$  Restart p  $\vee$  ( $\exists$ q m. a = Send p q m))
98   then (s p, a, s' p)  $\in$  part_trans p
99   else s' p = s p }"

100 definition Participants :: "'proc set  $\Rightarrow$  ('proc  $\Rightarrow$  't)  $\Rightarrow$  (('proc,'t msg)
     $\hookrightarrow$  action, 'proc  $\Rightarrow$  ('t,'proc) part_state) ioa" where
101 "Participants P t = (parts_asig P, { $\lambda$ p. PState (PInitial (t p)) {}},
102    $\hookrightarrow$  parts_trans P, {}, {})"

```

Um nun eine asynchrone Kommunikation zwischen Koordinator und Teilnehmern zu ermöglichen wird ein IOA zur Darstellung des Kommunikationsnetzes benötigt:

```

106 type_synonym ('proc,'t) chan_state = "('proc  $\times$  'proc  $\times$  't msg) set"
107
108 definition chan_trans :: " (('proc,'t) chan_state  $\times$  ('proc,'t msg)
     $\hookrightarrow$  action  $\times$  ('proc,'t) chan_state) set" where
109 "chan_trans =
110   {(M, Send s r m, M  $\cup$  {(s,r,m)}) | M s r m. True}
111    $\cup$  {(M, Receive s r m, M) | M s r m. (s,r,m)  $\in$  M}"
112
113 definition chan_asig :: " ('proc,'t) action signature" where
114 "chan_asig =
115   ({Send s r m | s r m. True},
116   {Receive s r m | s r m. True},
117   {})"
118

```

```

119 definition Channel :: "('proc,'t msg) action, ('proc,'t) chan_state)
    ↪ ioa" where
120 "Channel = (chan_asig, { {} }, chan_trans, {}, {})"

```

Nun kann das Zwei-Phasen-Commit-Protokoll als Komposition dieser drei IOA dargestellt werden:

```

124 definition System where
125 "System t P r = ((Coordinator 0 (t 0) (P ∪ {0}) r || Channel) ||
    ↪ Participants P t)"

```

5.4 Invarianten

Zum Beweis der einheitlichen Zustimmung mittels IOA werden die gleichen Invarianten benötigt, die auch für das Kleppmann Modell benötigt werden. Angepasst für das IOA Modell sieht die Invariante 1 wie folgt aus:

```

6 definition inv1 where
7   <inv1 s ↔
8     ((∃ msgs p. (snd s) p = PState PCommitted msgs) →
9       (∃ sender rcpt. (sender, rcpt, Commit) ∈ snd (fst s)))>

```

Die restlichen Invarianten können analog angepasst werden.

5.5 Beweis der Invarianten

Analog zum Kleppmannmodell dient der Beweis der Invariante 1 als Beispiel einer solchen Beweisführung. Der Beweis wird als Induktionsbeweis geführt.

```

369 lemma invariant1:
370   "invariant (System t (UNIV - {0}) r) inv1"
371 proof(induction rule: invariantI)

```

Zuerst wird gezeigt, dass Invariante 1 in allen Startzuständen gilt. Da jeder Teilautomat jeweils nur einen Startzustand hat, besitzt der IOA der Komposition auch nur einen Startzustand:

```

372 case (1 s)
373 moreover have "starts_of (System t (UNIV - {0}) r) = {((CState
    ↪ (CInitial (t 0) UNIV r) {},{}),λp. PState (PInitial (t p)) {}))}"
374 unfolding starts_of_def System_def par_def Coordinator_def
    ↪ Channel_def Participants_def by auto

```

Anschließend muss nur noch gezeigt werden, dass Invariante 1 in diesem Startzustand gilt, womit der Induktionsanfang abgeschlossen ist:

```

375 ultimately have "s = ((CState (CInitial (t 0) UNIV r) {}, {}), λp.
    ↪ PState (PInitial (t p)) {}))"
376 by auto
377 then show ?case
378 unfolding inv1_def by simp

```

Für den Beweis des Induktionsschritts muss gezeigt werden, dass bei jedem möglichen und erreichbaren Übergang des Automaten die Invariante erhalten bleibt:

```

380 case (2 s t a)
381 then show ?case
382 using invariant1_trans by metis

```

Dabei wird das folgende Lemma `invariant1_trans` zur Hilfe genommen. Es zeigt, dass die Invariante 1 auch nach einem gültigen Übergang gilt, solange der Ursprungszustand erreichbar war und die Invariante auch in diesem Zustand galt:

```

63 lemma invariant1_trans:
64   assumes "reachable (System t (UNIV - {0}) r) s"
65   and "inv1 s"
66   and "(s, a, s') ∈ trans_of (System t (UNIV - {0}) r)"
67   shows "inv1 s'"

```

Zuerst wird der Gesamtzustand des Automaten in die einzelnen Koordinator-, Teilnehmer- und Netzwerkzustände aufgeteilt:

```

68 proof -
69   obtain cstate cmsgs chmsgs pstates where decomp_s: "s = ((CState
    ↪ cstate cmsgs, chmsgs), pstates)"
70   by (metis coord_state.exhaust surj_pair)
71   obtain cstate' cmsgs' chmsgs' pstates' where decomp_s': "s' = ((CState
    ↪ cstate' cmsgs', chmsgs'), pstates')"
72   by (metis coord_state.exhaust surj_pair)

```

Anschließend wird durch die Anwendung des Hilfslemmas `System_trans_inv` festgehalten, dass jeder Teil des Gesamtautomaten bei diesem Übergang entweder selbst Teil dieses war oder sich sein Zustand nicht verändert hat.

Anschließend wird eine Fallunterscheidung über die getätigte Aktion geführt:

```

73 show ?thesis
74   using assms(3) unfolding decomp_s decomp_s'
75 proof (induction rule: System_trans_inv)
76   case System_Step
77   then show ?case
78   proof (cases a)

```

Sollte es sich um eine Start-Aktion handeln, so hat sich kein Zustand eines Teilnehmers geändert und es wurde keine Nachricht gesendet:

```

79   case (Start rcpt)
80   then have "pstates = pstates'"
81       using System_Step(3) unfolding parts_trans_def by auto
82   moreover have "chmsgs = chmsgs'"
83       using Start System_Step(2) by auto

```

Daraus folgt, dass sich die Gültigkeit der Invariante 1 bei dem Übergang nicht geändert hat. Da sie vorher galt, gilt sie auch nach dem Übergang:

```

84   ultimately have "inv1 s  $\longleftrightarrow$  inv1 s'"
85       using inv1_def by (metis assms(2) decomp_s decomp_s' fst_conv
86        $\hookrightarrow$  snd_conv)
87   then show ?thesis
88       using assms(2) decomp_s decomp_s' by simp

```

Sollte es sich um eine Send-Aktion handeln, so wurde lediglich eine Nachricht gesendet. Daraus folgt, dass sich kein Zustand eines Teilnehmers geändert haben kann:

```

89   case (Send sender rcpt _)
90   then have "( $\exists$ msgs p. pstates p = PState PCommitted msgs)  $\longleftrightarrow$ "
91        $\hookrightarrow$  "( $\exists$ msgs p. pstates' p = PState PCommitted msgs)"
92       by (metis System_Step.hyps(3) part_state.exhaust
93        $\hookrightarrow$  parts_trans_Send)

```

Da zusätzlich nach dem Übergang eine Commit-Nachricht existiert, wenn vorher eine solche Nachricht existierte, bleibt die Invariante 1 unverändert. Da sie vorher galt, gilt sie auch nach dem Übergang:

```

92   moreover have "( $\exists$ sender rcpt. (sender, rcpt, Commit)  $\in$  chmsgs)  $\longrightarrow$ "
93        $\hookrightarrow$  "( $\exists$ sender rcpt. (sender, rcpt, Commit)  $\in$  chmsgs')"
94       using Send System_Step(2) by auto
95   ultimately have "inv1 s  $\longleftrightarrow$  inv1 s'"
96       using inv1_def by (metis assms(2) decomp_s decomp_s' fst_conv
97        $\hookrightarrow$  snd_conv)
98   then show ?thesis
99       using assms(2) decomp_s decomp_s' by simp

```

Sollte es sich um eine Receive-Aktion handeln, so wurde keine Nachricht gesendet und es wird eine Fallunterscheidung bezüglich des Empfängers durchgeführt:

```

99   case (Receive sender rcpt msg)
100   have chEquiv:"chmsgs = chmsgs'"
101       using Receive System_Step(2) by blast
102   moreover have receive:"(sender, rcpt, msg)  $\in$  chmsgs'"
103       by (metis Receive decomp_s decomp_s' System_Receive_chan_step
104        $\hookrightarrow$  assms(3))
105   then show ?thesis
106   proof(cases "rcpt = 0")

```


Sollte es sich bei dem Empfänger der Nachricht um den Koordinator handeln, ändert sich kein Zustand eines Teilnehmers und die Invariante bleibt unverändert:

```

106     case True
107     then have "pstates = pstates'"
108         using True System_Step(3) Receive unfolding parts_trans_def by
            ↪ auto
109     then have "inv1 s  $\longleftrightarrow$  inv1 s'"
110         using inv1_def chEquiv by (metis decomp_s decomp_s' fst_conv
            ↪ snd_conv)
111     then show ?thesis
112         using assms(2) decomp_s decomp_s' by simp

```

Sollte es sich stattdessen um einen Teilnehmer handeln, so kann lediglich ein Teilnehmer diesen Übergang gemacht haben:

```

114     case False
115     then have otherPartsEquiv: " $\forall p \neq rcpt. pstates\ p = pstates'\ p$ "
116         using System_Step(3) Receive unfolding parts_trans_def by auto

```

Nun kann unterschieden werden, ob sich der Zustand des Empfängers geändert hat. Ist dies nicht der Fall, so hat sich kein Zustand eines Empfängers geändert und der Beweis ist trivial:

```

118     proof(cases "pstates rcpt = pstates' rcpt")
119     case True
120     then have "pstates = pstates'"
121         using otherPartsEquiv by auto
122     then show ?thesis
123         using chEquiv assms(2) inv1_def decomp_s decomp_s' by (metis
            ↪ fst_conv snd_conv)

```

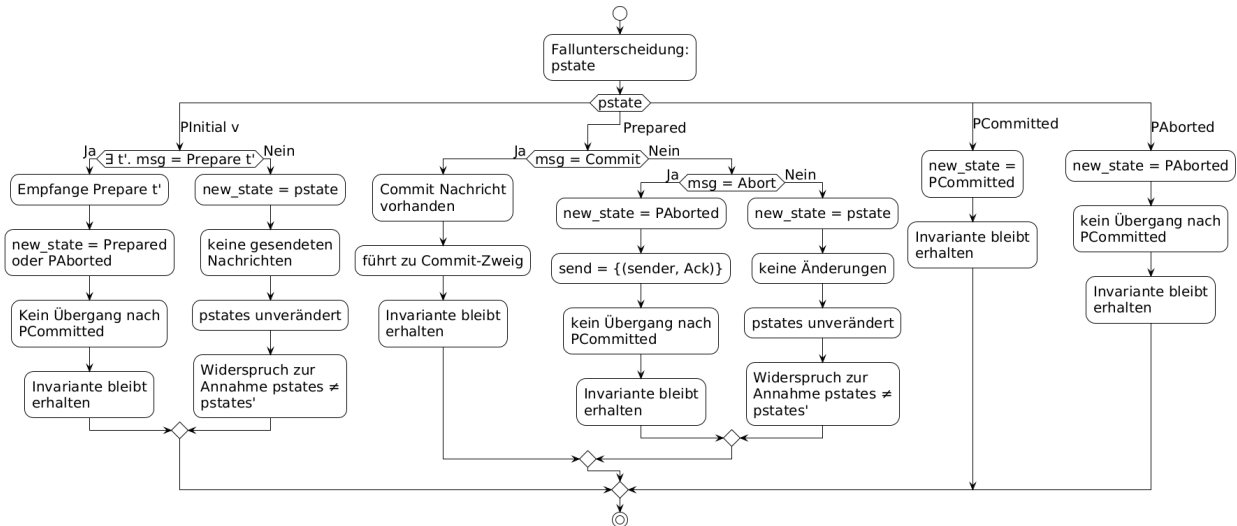
Sollte sich der Zustand geändert haben, so muss es zusammen mit den gesendeten Nachrichten einen entsprechenden Übergang in der Übergangsfunktion eines Teilnehmers geben:

```

125     case False
126     then obtain pstate pmsgs where stateDecomp: "pstates rcpt =
            ↪ PState pstate pmsgs"
127         using part_state.exhaust by blast
128     then obtain new_state sent where step: "participant_step rcpt
            ↪ pstate a = (new_state, sent)"
129         using System_Step(3) False by fastforce
130     have update_eq: "pstates' rcpt = PState new_state (pmsgs  $\cup$ 
            ↪ (( $\lambda$ msg. (rcpt, msg)) ` sent))"
131         by (smt (verit, ccfv_SIG) step parts_trans_proj stateDecomp
            ↪ parts_trans_step_result System_Step(3) False Receive)

```

Abbildung 2: Auszug der schematischen Beweisstruktur für invariant1_trans



Dank dieser Unterteilung in alter Zustand, neuer Zustand und empfangene und gesendete Nachrichten, kann der Rest des Beweises analog zum Beweis der Invarianten 1 im Modell von Kleppmann geführt werden. Das schematische Vorgehen ist in Abbildung 2 beschrieben.

Im gleichen Stile können die Fälle für eine Timeout- und Restart-Aktion gezeigt werden.

5.6 Beweis der einheitlichen Zustimmung

Die Beweisführung der einheitlichen Zustimmung erfolgt mittels der gleichen Invarianten analog zum Modell von Kleppmann. Aufgrund von besserer Übersichtlichkeit, werden folgende Definitionen eingeführt, um anzugeben ob sich ein Prozess im Committed- oder Aborted-Zustand befindet:

```

10 definition process_committed where
11   "process_committed p s  $\longleftrightarrow$ 
12     (if p = 0 then ( $\exists$  a ack cmsgs. fst (fst s) = CState (CCommitted a
13        $\hookrightarrow$  ack) cmsgs)
14     else ( $\exists$  m. (snd s) p = PState PCommitted m))"
15 definition process_aborted where
16   "process_aborted p s  $\longleftrightarrow$ 
17     (if p = 0 then ( $\exists$  a ack cmsgs. fst (fst s) = CState (CAborted a ack)
18        $\hookrightarrow$  cmsgs)
19     else ( $\exists$  m. (snd s) p = PState PAborted m))"

```

Diese Definitionen erlauben eine analoge Formulierung der folgenden Lemmas im Vergleich zum Modell von Kleppmann. Die Beweisführung bleibt semantisch unverändert:

```

20 lemma ac1_NoCoordinator:
21   assumes "reachable (System t (UNIV - {0}) r) s"
22   and "proc1  $\neq$  0  $\wedge$  proc2  $\neq$  0"
23   and "process_committed proc1 s  $\vee$  process_aborted proc1 s"
24   and "process_committed proc2 s  $\vee$  process_aborted proc2 s"
25   shows "process_committed proc1 s  $\longleftrightarrow$  process_committed proc2 s"
68 lemma ac1_OneCoordinator:
69   assumes "reachable (System t (UNIV - {0}) r) s"
70   and "p  $\neq$  0 "
71   and "process_committed 0 s  $\vee$  process_aborted 0 s"
72   and "process_committed p s  $\vee$  process_aborted p s"
73   shows "process_committed 0 s  $\longleftrightarrow$  process_committed p s"
107 lemma ac1_Theorem:
108   assumes "reachable (System t (UNIV - {0}) r) s"
109   and "process_committed proc1 s  $\vee$  process_aborted proc1 s"
110   and "process_committed proc2 s  $\vee$  process_aborted proc2 s"
111   shows "process_committed proc1 s  $\longleftrightarrow$  process_committed proc2 s"

```

6 Vergleich der Modellierungsansätze

Da nun das Zwei-Phasen-Commit-Protokoll in beiden Frameworks implementiert und die zentrale Eigenschaft der einheitlichen Zustimmung bewiesen wurde, können darauf aufbauend die verwendeten Modellierungsansätze detailliert gegenübergestellt werden.

6.1 Erarbeitung der Vergleichskriterien

Für einen aussagekräftigen Vergleich werden rein quantitative Metriken, wie etwa die Anzahl der Beweiszeilen, bewusst vernachlässigt, da diese stark von individuellen Modellierungsentscheidungen abhängen und nicht zwingend die Qualität eines Ansatzes widerspiegeln. Stattdessen fokussiert sich dieser Vergleich auf qualitative Kriterien, welche maßgeblich bestimmen, wie skalierbar, anwendbar und nutzerfreundlich ein Framework in der Praxis ist:

Ein flexibles Framework kann zur Modellierung verschiedenster Systemarchitekturen und Fehlermodelle genutzt werden, anstatt den Nutzer in vorgefertigte Schablonen zu zwingen [13]. Modularität wiederum ist entscheidend für das Prinzip 'Teile und Herrsche': Erlaubt ein Framework die Kapselung und Wiederverwendung von Systemkomponenten, lassen sich auch gigantische Zustandsräume beherrschen.

Die Nähe zur Spezifikation ist essenziell für die Fehlerprävention. Wenn eine Formalisierung zu weit vom eigentlichen Algorithmus abstrahiert ist, können beim Übergang unbemerkt semantische Fehler entstehen. Es ist von Vorteil, wenn eine Formalisierung nicht zu weit vom eigentlichen Algorithmus abweicht, da andernfalls leichter unbemerkt semantische Fehler entstehen können. Dabei hilft ein Vergleich der letztendlichen Darstellung des Zwei-Phasen-Commit-Protokolls, um ein Verständnis über die Ähnlichkeit beider Frameworks zueinander aufzubauen.

Weiterhin hilft die Beweisstruktur und der Beweisaufwand die praktische Nutzbarkeit einzuschätzen. Ein mathematisch perfektes Modell bietet wenig Nutzen, wenn die formale Verifikation selbst einfachster Eigenschaften sehr aufwändig ist.

Ein gutes Framework sollte über reine Safety-Garantien hinausgehen. Die Unterstützung von Liveness-Eigenschaften ist ein wichtiges Kriterium, da sie den Nachweis ermöglicht, dass ein verteiltes System tatsächlichen Fortschritt macht und nicht etwa in einem Dead-lock verharrt [5].

Der Einstieg in die formale Verifikation ist zumeist komplex. Dokumentationen und existierende Code-Beispiele sind dabei sehr hilfreich, um diese Einstiegshürde zu senken. Zudem sollte Isabelle's Automatisierung vom Framework gut genutzt werden können um den Beweissprozess drastisch zu verkürzen und zu vereinfachen.

6.2 Flexibilität und Modularität

Die beiden Frameworks verfolgen bei der Flexibilität fast entgegengesetzte Philosophien. Die Ausführungssemantik im Modell von Kleppmann ist sehr streng: Es ist fest vorgegeben, dass immer exakt ein Prozess ein Event ausführt [5]. Globale Events sind nicht ohne

Umwege möglich und die Struktur des Netzwerks ist durch den globalen Nachrichtenpool starr vorgegeben. Das IOA-Framework hingegen zeichnet sich durch seinen inhärenten Nicht-Determinismus aus. Auch die Netzwerkkanäle müssen selbst modelliert werden. Diese enorme Flexibilität erlaubt es, verschiedenste Netzwerktopologien oder spezifische Kommunikationsmuster präzise abzubilden.

Das Modell von Kleppmann sieht Modularität nativ nicht vor [6]. Zwar lassen sich Übergangsfunktionen manuell kombinieren, wie es bei Koordinator und Teilnehmer gemacht wurde, das System bleibt jedoch tendenziell eine starre monolithische Einheit. Das formale IOA Modell von Lynch hingegen hat die Komposition als einen Hauptbestandteil [6], welche auch nativ vom Isabelle-Framework unterstützt wird. Eine Einschränkung besteht lediglich darin, dass das Framework eine Komposition von unendlich vielen IOA nicht nativ unterstützt und wie bei dem Teilnehmer-IOA eigenständig durchgeführt werden muss.

6.3 Ähnlichkeit Formalisierung vs Algorithmus

Im Modell von Kleppmann konnten die Übergangsfunktionen fast vollständig und direkt der Protokollspezifikation nachempfunden werden, weshalb sich in diesem Modell, Formalisierung und Algorithmus/Spezifikation sehr ähneln. Im IOA-Framework sind die Übergangsfunktionen selbst zwar ebenfalls ähnlich, jedoch ist die daraus resultierende Übergangsrelation deutlich abstrakter, da sie für die Darstellung von Nicht-Determinismus genutzt wird. Dies sorgt für eine stärkere Abstraktion vom ursprünglichen Algorithmus.

6.4 Formalisierung des Zwei-Phasen-Commit-Protokolls

Beim Modell von Kleppmann orientieren sich die Übergangsfunktionen stark an der Spezifikation der Protokollbeschreibung. Um die Invarianten nicht unnötig umständlich formulieren zu müssen, wurde für die Zustände von Koordinator und Teilnehmern ein gemeinsamer Datentyp verwendet.

Im IOA-Framework ist der Modellierungsaufwand merklich höher: Neben den Übergangsfunktionen, die ähnlich direkt aus der Protokollspezifikation hervor gingen, müssen Aktions-Signaturen definiert und explizite Übergangsrelationen generiert werden. Ein wesentlicher Faktor für den Mehraufwand ist zudem die explizite Modellierung der Netzwerkkanäle als eigener IOA und die anschließende Komposition aller Teilautomaten.

6.5 Beweisstruktur

Die grundsätzliche Struktur der Beweise ähnelt sich stark: Der Beweis von Invarianten erfordert in beiden Frameworks eine Induktion über die Ausführung in Verbindung einer anschließenden Fallunterscheidung, wer den Übergang getätigt hat gefolgt von ausgeführter Aktion/ ausgeführtem Event und aktuellem Zustand des ausführenden Prozesses. Da das Modell von Kleppmann deutlich strenger ist entfallen viele Zwischenschritte, die beim potenziell Nicht-Deterministischen IOA-Modell nötig waren. Der Beweis der einheitlichen Zustimmung ist in beiden Modellen nahezu identisch.

6.6 Beweisaufwand

Der Beweisaufwand korreliert stark mit der Flexibilität des Frameworks. Das Modell von Kleppmann ist deterministischer und strukturell weniger flexibel, was tendenziell zu deutlich kürzeren und direkteren Beweisen führt.

Das IOA Modell hingegen ist inhärent nicht-deterministisch und in seiner Struktur enorm flexibel. Diese schwachen strukturellen Vorgaben erfordern mehr manuellen Aufwand, da die Beweisführung direkter gelenkt werden muss.

6.7 Nachweis von Liveness-Eigenschaften

Während Safety-Eigenschaften, wie einheitliche Zustimmung garantieren, dass 'nichts Ungewünschtes' passiert, stellen Liveness-Eigenschaften sicher, dass 'irgendwann das Erwünschte' passiert [3]. Dies steht oft in engem Zusammenhang mit Annahmen über Fairness oder das Netzwerk.

Kleppmann macht bewusst keine Annahmen über Fairness, da diese oft zu fallspezifisch sind. Dadurch unterstützt das Modell von Kleppmann Fairness nicht nativ. Lässt sich aber über eigene Annahmen in Kombination mit der Event-Semantik gut abbilden.

Das IOA Modell von Lynch hingegen unterstützt Fairness grundlegend. Auch das verwendete Framework bietet grundlegende Funktionalität für Fairness. Da das Netzwerk bei IOA als eigenständiger Automat modelliert werden muss, können realistische Netzwerkannahmen direkt über die Komposition in das Gesamtsystem integriert werden.

6.8 Automatisierungsgrad in Isabelle

Ein großer Vorteil der computergestützten Beweisführung in Isabelle ist die Automatisierung durch Hilfen wie 'Sledgehammer' [8]. Kleppmanns Framework ist klein und auf das Wesentliche reduziert, was sehr gut mit Isabelles Automatisierung einher geht. Sledgehammer bietet hier gute Unterstützung.

Im IOA-Framework fällt es Sledgehammer aufgrund der vielen, tief verschachtelten Definitionen schwer, eigenständig Beweise zu finden. Oft müssen Definitionen erst manuell expandiert werden. Dies führt außerdem dazu, dass die zu zeigenden Aussagen deutlich größer und unübersichtlicher werden, weshalb für das IOA-Framework weitaus öfter manuell Hilfslemmata, insbesondere für Übergänge innerhalb von Kompositionen, geschrieben werden müssen.

6.9 Dokumentation zur Nutzung der Frameworks

Gute Beweisbeispiele und Dokumentationen sind essenziell für den Einstieg in das jeweilige Framework. Für das Modell von Kleppmann existiert eine zugängliche Demonstration von Kleppmann selbst, in der ein einfacher Konsensalgorithmus formalisiert wird [5]. Der Code ist online verfügbar und leicht verständlich [10]. Bei sehr komplexen Systemen kann es jedoch passieren, dass man aufgrund fehlender Werkzeuge 'kreativ' werden muss, um das Protokoll abzubilden.

Da I/O-Automaten ein theoretisch weit verbreitetes Modell sind, gibt es viel konzeptionelle Literatur. Zwar existieren auch Formalisierungsbeispiele für ähnliche IOA-Frameworks (etwa von Nipkow selbst), diese zeigen jedoch oft wenig konkreten Code zur Beweisführung, was die Einarbeitung in die praktische Nutzung erschwert [14] [9].

7 Schluss

Die Verwendung formaler Beweismethoden zum Nachweis von Korrektheit verteilter Systeme ist besonders wichtig, da solche Systeme oft zu komplex sind und Fehler teilweise lange unentdeckt bleiben [5]. Die Darstellung eines solchen verteilten Systems kann über eine Vielzahl an Modellen geschehen.

In dieser Arbeit wurde das Zwei-Phasen-Commit-Protokoll in Isabelle mithilfe zweier unterschiedlicher Frameworks modelliert und die Eigenschaft der einheitlichen Zustimmung bewiesen. Dabei wurden typische Fehlerquellen verteilter Systeme wie Nachrichtenverlust, Duplikation oder Prozessausfälle berücksichtigt.

Anschließend erfolgte ein Vergleich beider Frameworks sowie der Modellierung und Beweisführung. Der Vergleich zeigt, dass beide Ansätze grundsätzlich geeignet sind, verteilte Systeme formal zu beschreiben und deren Korrektheit zu verifizieren, jedoch unterschiedliche Schwerpunkte setzen. Das Modell von Kleppmann bietet eine praxisnahe und einfache Sicht auf verteilte Systeme, mit der die Automatisierung in Isabelle voll ausgenutzt werden kann. Das IOA-Framework stammt von einem weit verbreiteten mathematischen Modell, das von Nancy A. Lynch und Mark R. Tuttle erstmals eingeführt wurde. Es überzeugt besonders mit seiner Flexibilität in Modellierung und Komposition mehrerer IOA, was den Wiederverwendungswert aber auch die Komplexität von Beweisen in diesem Modell erhöht.

Ein wesentlicher Unterschied zeigt sich zudem in der Unterstützung weicherer Eigenschaften. Während das Modell von Kleppmann primär auf Safety-Eigenschaften ausgelegt ist und Fairnessannahmen nur indirekt integriert werden können, bietet das IOA-Modell hierfür bereits konzeptionelle Grundlagen. Dadurch eignet es sich besser für die Analyse von Liveness-Eigenschaften und langfristigem Systemverhalten. Zusätzlich verfügt das Modell von Kleppmann über eine gut dokumentierte beispielhafte Nutzung des Frameworks, was den Einstieg damit deutlich vereinfacht. Gleichzeitig können die von Isabelle bereitgestellten Hilfen wie 'Sledgehammer' deutlich besser mit dem Modell von Kleppmann als mit dem IOA-Modell genutzt werden.

Zusammenfassend lässt sich sagen, dass sich beide Ansätze dazu eignen die Korrektheit verteilter Systeme auch unter schwierigen Bedingungen zuverlässig nachzuweisen. Gleichzeitig hat die Wahl der Modellierungsmethode einen erheblichen Einfluss auf die Komplexität der Verifikation und sollte bewusst getroffen werden.

8 Quellen

Literatur

- [1] *How one line of code caused a \$60 million loss*. 13. Nov. 2023. URL: <https://read.engineerscodex.com/p/how-one-line-of-code-caused-a-60> (besucht am 07. 04. 2026).
- [2] *The Crash of the AT&T Network in 1990*. 26. Feb. 1990. URL: <https://telephoneworld.org/landline-telephone-history/the-crash-of-the-att-network-in-1990/> (besucht am 12. 04. 2026).
- [3] Martin Kleppmann. *Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems*. 11. Juli 2016. URL: [https://0-lucas.github.io/digital-garden/99.-Books/Martin-Kleppmann---Designing-Data-Intensive-Applications_-0%E2%80%99Reilly-Media-\(2017\).pdf](https://0-lucas.github.io/digital-garden/99.-Books/Martin-Kleppmann---Designing-Data-Intensive-Applications_-0%E2%80%99Reilly-Media-(2017).pdf) (besucht am 06. 04. 2026).
- [4] Jens Lechtenbörger. *2-Phase-Commit Protocol*. type. Universität Münster. URL: <http://www.lgis.informatik.uni-kl.de/cms/fileadmin/courses/SS2008/Informationssysteme/Addons/2PC.pdf> (besucht am 04. 04. 2026).
- [5] Martin Kleppmann. *Verifying distributed systems with Isabelle/HOL*. 12. Okt. 2022. URL: <https://martin.kleppmann.com/2022/10/12/verifying-distributed-systems-isabelle.html> (besucht am 12. 04. 2026).
- [6] Nancy A. Lynch. *Distributed Algorithms*. 16. Apr. 1996. URL: <https://archive.org/details/distributedalgor0000lync/page/208/mode/2up> (besucht am 06. 04. 2026).
- [7] Marco Canini. *Two-Phase Commit*. CS 240: Computing Systems and Concurrency Lecture 10. URL: <https://sands.kaust.edu.sa/classes/CS240/F19/docs/L10-2pc.pdf> (besucht am 12. 04. 2026).
- [8] Gerwin Klein Tobias Nipkow. *Concrete Semantics. with Isabelle/HOL*. 21. Jan. 2026. URL: <http://www.concrete-semantics.org/concrete-semantics.pdf> (besucht am 08. 04. 2026).
- [9] Tobias Nipkow Olaf Müller. *Traces of I/O Automata in Isabelle/HOLCF*. type. Technischen Universität München. (Besucht am 12. 04. 2026).
- [10] Martin Kleppmann. *Correctness proofs of distributed systems with Isabelle*. URL: <https://gist.github.com/ept/b6872fc541a68a321a26198b53b3896b> (besucht am 12. 04. 2026).
- [11] *Input/output automaton*. URL: https://en.wikipedia.org/wiki/Input/output_automaton (besucht am 04. 12. 2026).
- [12] Tobias Nipkow Olaf Müller Konrad Slind. *Theory Automata*. URL: <https://isabelle.in.tum.de/library/HOL/IOA/Automata.html> (besucht am 04. 12. 2026).

- [13] Franziska Beeler. *What Makes an Effective Framework?* 29. Juli 2024. URL: <https://www.franziskabeeler.com/post/what-makes-an-effective-framework>.
- [14] Konrad Slind Tobias Nipkow. *I/O Automata in Isabelle/HOL*. type. Technischen Universität München. (Besucht am 12. 04. 2026).